Open-Set Encrypted Traffic Recognition: A Multi-View Framework with Global and Local Empirical Support

***Abstract*—**

I. Introduction

The growth of cloud services, mobile applications, and the Internet of Things has diversified network applications and communication behaviors, making accurate traffic identification increasingly important for network management. Encrypted-traffic classification uses observable communication features to support application identification, bandwidth allocation, quality-of-service assurance, and anomalous-communication monitoring [1]–[3]. As encryption limits plaintext matching, research has shifted toward learning traffic features from observable packet bytes, lengths, directions, and timing [1]–[3]. Deep Fingerprinting shows, in the Tor website-fingerprinting setting, that deep models can identify visited websites from communication patterns even when payloads are encrypted [4]. Pretrained models, hierarchical representations, and graph neural networks improve classification by capturing byte context, packet organization, and communication interactions [1]–[3], [5]–[7].

Although stronger closed-set classifiers can improve open-set detection [8], **accurate known-class classification does not directly ensure reliable unknown-class rejection.** Applications and attacks evolve in operational networks, and closed-set classifiers may assign traffic unseen during training to an existing class [9]–[11]. Foundational open-set recognition research highlights the need to limit open space risk, where regions far from known training samples are assigned known-class labels [12]. OpenMax further recalibrates deep activation patterns to reserve probability for unknown classes [13]. In the traffic domain, existing studies supplement class prediction with unknown-detection criteria based on discriminative boundaries or known-class distribution modeling [10], [11], [14], [15]. These studies provide a foundation for open-set traffic recognition. We focus on learning and calibration using only known-class data and examine two issues: **complementarity in traffic representations** and **the adequacy of rejection criteria**.

(1) Complementary traffic representations for unknown-class rejection. Unknown-class rejection requires representations that distinguish known classes while revealing differences between unseen samples and known classes. Byte sequences, packet interactions, and multilevel byte organization capture different aspects of communication [2], [3], [6]. Unknown traffic may resemble known classes in some features while differing in others. If a representation relies too heavily on a particular view, differences captured by other views may be overlooked, making some unknown samples harder to distinguish from known classes. An important question is therefore how to integrate complementary information from different views, using only known-class training data, to provide a sufficient representation basis for unknown-class rejection.

(2) Limited characterization of local sample distributions by class prototypes. Generative methods model known-class distributions to detect unknown traffic. Open-Detect [11] learns Gaussian class prototypes through generative and discriminative constraints, captures class location and dispersion, and rejects samples by prototype distance. However, prototypes summarize global distributions without directly examining nearby training samples. With uneven or multimodal class distributions, samples satisfying the prototype-distance acceptance criterion may still lie in sparsely sampled regions; proximity to the global distribution need not imply sufficient local support. Prior work uses Mahalanobis distance under class-conditional Gaussian models [16] and nonparametric deep nearest-neighbor distance [17] to measure deviation from known data, providing distribution-based and neighborhood-

based criteria. Combining global class position with within-class neighbors can therefore provide complementary evidence for unknown rejection.

To address these issues, we propose TriGLE (Tri-view Global-Local Empirical support framework) for open-set encrypted-traffic recognition. To construct complementary representations for unknown rejection, TriGLE fuses byte sequences, packet interactions, and multilevel byte organization into a unified traffic representation. In this space, empirical class centroids provide a global location reference, while within-class nearest neighbors assess support from actual training samples, yielding rejection criteria that account for both global and local information. Representation learning and reference-set construction use only known-class samples, and rejection thresholds are determined from known-class validation data, supporting known-class classification and unknown-class rejection under this training condition.

Our contributions are as follows:

1) **A unified three-view traffic representation.** We jointly learn byte context, packet interactions, and multilevel byte organization, using Gaussian adaptation and deterministic fusion to provide a shared representation for known-class classification and unknown-class rejection.
2) **Unknown rejection through global and local empirical support.** We combine empirical centroid distance with within-class neighbor distance and use known-class validation data for distance normalization and threshold calibration, assessing unknown traffic through both global class position and local sample neighborhoods.
3) **Evaluation on multiple public datasets.** We evaluate known-class classification, unknown-class rejection, and component contributions on ISCX-VPN, ISCX-Tor, USTC-TFC2016, CIC-IDS2017, and VNAT. Across four VNAT complete-service holdout folds, TriGLE achieves mean AUROC of 99.22% and UFAR of 1.09%. On identical representations, adding local support raises AUROC from 97.43% with empirical centroids alone to 99.22%, providing experimental evidence for the complementary roles of global and local information.

The remainder of this paper is organized as follows. Section II reviews encrypted-traffic classification and unknown-traffic detection. Section III presents the problem formulation, three-view representation learning and fusion, and global and local empirical-support decision criteria. Section IV describes the experimental setup and evaluates known-class classification and unknown-class rejection performance. Section V concludes the paper and discusses limitations and future directions.

II. Related Work

A. Encrypted-Traffic Classification

Encrypted-traffic classifiers learn representations that distinguish traffic classes using observable packet attributes such as bytes, lengths, directions, and timing. Sequence models capture temporal patterns and relationships across packets. FS-Net [18] represents flows through packet-length and status sequences before classification. ET-BERT [1] pretrains contextualized datagram representations on large volumes of unlabeled traffic and fine-tunes them for downstream classification. TrafficFormer [2] combines fine-grained multiclass pretraining with packet-field augmentation to improve traffic representations. MIETT [5] treats each flow as a set of packet instances and models both within-packet and cross-packet relationships through two-level attention, packet position prediction, and flow-level contrastive learning.

Hierarchical and graph-based approaches capture additional aspects of traffic structure. YaTC [6] arranges packet headers and payloads in multilevel matrices and combines packet-level and flow-level attention with masked pretraining to preserve byte organization and cross-packet structure. GraphDApp [3] models bidirectional communications as flow interaction graphs and identifies decentralized applications with a graph neural network. TFE-GNN [7] uses pointwise mutual information to build byte association graphs, encodes headers and payloads separately, and combines them through cross-gated fusion. MH-Net [19] uses multigranular heterogeneous graphs to model headers, payloads, and their relationships jointly. DigTraffic [20] constructs message-aware flow interaction graphs and uses a dual-channel graph Transformer to fuse length and timing features. These approaches capture different aspects of hierarchical organization, communication interactions, and byte associations. Deep multiview learning also explores joint representations through cross-view correlation and reconstruction objectives [21], motivating the use of information from multiple views.

Most of these classifiers target predefined classes and do not explicitly reject unknown classes [1], [2], [5]–[7], [19]. Accurate discrimination among known classes does not ensure reliable detection of unknown traffic. GraphDApp and DigTraffic both include open-world evaluations. GraphDApp includes unmonitored traffic in training [3], while DigTraffic primarily evaluates binary discrimination between seen and unseen traffic [20]. Maintaining fine-grained classification while reliably rejecting unknown classes using only known-class training data remains an open problem.

B. Unknown-Traffic Detection Methods

Unknown-traffic detection aims to classify known traffic accurately and reject traffic from classes absent during training. The main approaches use discriminative or generative modeling, with some methods also supporting class discovery and incremental updates.

1) Discriminative Unknown-Traffic Detection: Discriminative methods separate known and unknown traffic using classifier outputs, partitions of feature space, or feature distances. MSP [22] and Energy [23] derive detection scores from softmax probabilities and logits, respectively. ViM [24] combines feature-space residuals with logits, illustrating how different detection cues can complement one another. Zhang et al. [9] use classification confidence to screen candidate unknown samples and update the classifier through a self-learning discriminator, clustering, and automatic labeling. FEC-OSL [10] combines dual-branch feature extraction with energy boundary learning to distinguish known and unknown traffic. It uses auxiliary unknown data and deep clustering to separate unknown

traffic into finer categories. FOSS [25] uses isolation trees to partition feature space, detect local anomalies, and distinguish unknown attacks. It adapts to changes in legitimate traffic through incremental updates. RFG-HELAD [26] combines latent features from contrastive learning and a generative adversarial network. It uses nearest-neighbor distances in the learned feature space to detect unknown attacks and a heterogeneous ensemble to improve robustness. RoNeTC [27] combines multiview features with uncertainty-aware classification. FlowPrint [28] instead constructs application fingerprints from temporal correlations among destination-related features and detects previously unseen applications. These methods extend detection criteria beyond classification confidence to spatial structure, sample neighborhoods, and traffic correlations. Their requirements for auxiliary data and model updates differ.

2) Generative Unknown-Traffic Detection: Generative methods use reconstruction or probability distributions to model known classes and detect test samples that deviate from those patterns. Trident [14] uses multiple autoencoder-based one-class learners and class-specific anomaly thresholds to reject unknown samples. Clustering supports new-class discovery and incremental updates. Yang et al. [15] combine a conditional variational autoencoder with extreme value theory to identify unknown attacks from reconstruction errors and their tail distributions. They also cluster benign traffic to reduce false alarms. Open-Detect [11] uses Gaussian class prototypes with generative and discriminative constraints to improve within-class compactness and between-class separation. It rejects unknown samples according to their distances from the prototypes. Reconstruction and distribution modeling provide complementary criteria for unknown-traffic detection. Class-conditional Gaussian models with Mahalanobis distances [16] and nonparametric deep nearest-neighbor distances [17] further illustrate distribution-based and sample-based detection criteria. Class prototypes capture global structure, while nearest-neighbor measures capture support from local training samples. Combining these signals under known-class-only training remains an open direction.

C. Summary

Prior studies have improved encrypted-traffic classification and unknown-traffic detection through different representations and decision criteria. One open direction is to fuse byte sequences, packet interactions, and multilevel byte structure while combining global class distributions with local neighborhood information to support known-class classification and unknown-class rejection.

III. Method

TriGLE combines representation learning with global and local empirical support for open-set encrypted-traffic recognition. The method characterizes traffic through byte sequences, packet interactions, and multilevel byte organization, and constructs a unified representation through Gaussian adaptation and deterministic fusion. It then combines empirical class centroids with within-class sample neighborhoods to measure deviations from known classes in terms of both global position and local neighborhood relationships. The resulting rejection score integrates global and local information for known-class classification and unknown-class rejection. Fig. 3-1 presents the overall framework.

TABLE 3-1
Main Notation and Definitions

Symbol	Definition
$\mathcal{C}_K, \mathcal{C}_U$	Known- and unknown-class sets
$\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{val}}$	Known training and validation sets
$z_v(x), \mu_v(x)$	Encoded view features and latent mean after Gaussian adaptation
$h(x), \ell(x)$	Fused features and known-class classification score vector
$\mathcal{T}_c, b_c$	Training representation set and empirical centroid of class c
$c^*(x)$	Class of the nearest centroid
$d_G(x), d_L(x)$	Global centroid distance and local nearest-neighbor distance
$s(x), \tau$	Combined rejection score and rejection threshold

A. Problem Definition and Notation

Following the open-set recognition setting [12], the labeled training set for encrypted traffic contains N samples, $\mathcal{D}_{\text{tr}} = \{(x_i, y_i)\}_{i=1}^{N}$. Here, $x_i \in \mathcal{X}$ denotes a bidirectional flow, $\mathcal{X}$ is the traffic sample space, and $y_i \in \mathcal{C}_K = \{1, \ldots, C\}$ is its class label, with C known classes. The known-class validation set is $\mathcal{D}_{\text{val}}$, whose labels also belong to $\mathcal{C}_K$.

At test time, the model processes traffic from both the known-class set $\mathcal{C}_K$ and the unknown-class set $\mathcal{C}_U$. Unknown classes appear in neither the training set nor the validation set, and $\mathcal{C}_K \cap \mathcal{C}_U = \varnothing$. Test samples therefore extend beyond the classes observed during training.

Our objective is to learn an open-set recognition model f that maps an input flow to a known-class label or a unified unknown label:

$$f : \mathcal{X} \to \mathcal{C}_K \cup \{\perp\}, \tag{1}$$

Here, $\perp$ denotes the unknown label. The learning objective has two parts: (1) accurately distinguish flows in $\mathcal{C}_K$ and output their known-class labels; and (2) identify flows in $\mathcal{C}_U$ and assign them the unified unknown label. The model thus addresses both known-class classification and unknown-class rejection. Table 3-1 summarizes the main notation; other symbols are defined at first use.

B. Traffic Representation Learning in TriGLE

Encrypted-traffic class characteristics appear in byte combinations and contextual relationships, interactions between packets, and the internal hierarchical structure of packets. We construct byte-sequence, flow-interaction-graph, and byte-image views to learn byte-context dependencies, packet interactions, and multilevel byte organization, respectively. Let $x = \{p_1, p_2, \ldots, p_{n_x}\}$ denote the chronologically ordered packet sequence of a bidirectional flow, where n_x is the packet count. The sending direction of the first packet defines the forward direction. Each branch constructs its input from the same flow and extracts complementary flow-level features for subsequent Gaussian adaptation and fusion.

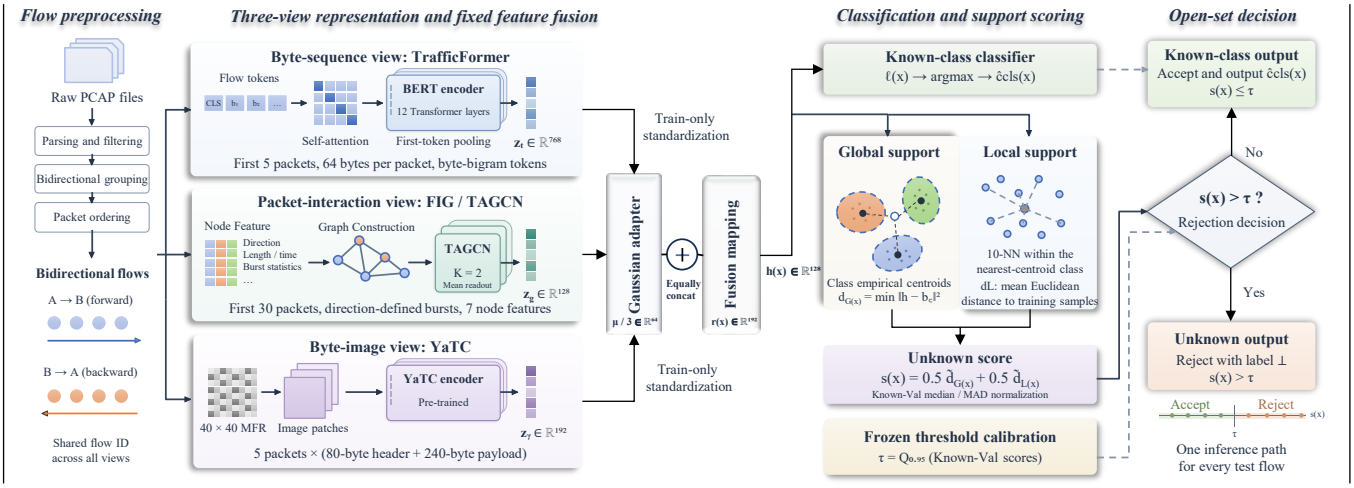


Fig. 3-1. Overall framework of the proposed method.

1) Byte-Sequence Branch: The byte-sequence branch captures local byte-combination patterns and contextual dependencies along the packet-byte sequence. We use pretrained TrafficFormer [2] as the byte-sequence encoder to map raw packet bytes to a flow-level sequence representation.

For the chronologically ordered flow $x = \{p_1, p_2, \ldots, p_{n_x}\}$, we select the first $\min(5, n_x)$ packets. After removing the Ethernet header, we retain at most 64 bytes per packet and form hexadecimal byte bigrams following the TrafficFormer input representation. For a byte sequence $(b_1, b_2, \ldots, b_m)$, the bigram representation is $(b_1\|b_2, b_2\|b_3, \ldots, b_{m-1}\|b_m)$, where $\|$ denotes concatenation of adjacent bytes. TrafficFormer's token mapping converts these combinations into discrete tokens, which form a flow-level sequence in the original packet order. After adding the [CLS] token, we truncate or pad the input to a maximum of 320 tokens.

Let $T(x)$ denote the resulting token sequence. We extract features using the pretrained TrafficFormer encoder and fine-tune it with supervision on the known-class training set. The hidden representation at the [CLS] position in the final encoder layer serves as the flow-level byte-sequence feature:

$$z_t(x) = E_{\mathrm{TF}}(T(x))_{\mathrm{CLS}} \in \mathbb{R}^{768}, \tag{2}$$

Here, E_{TF} is the fine-tuned TrafficFormer encoder, and $z_t(x)$ is the 768-dimensional byte-sequence representation used by the subsequent multiview adaptation and fusion module.

2) Flow-Interaction-Graph Branch: The flow-interaction-graph branch characterizes communication behavior through relationships between packets, complementing the low-level content features of the byte-sequence branch. Following existing Flow Interaction Graph (FIG) modeling approaches [3], [10], we represent a bidirectional flow as a packet-level graph. A Topology Adaptive Graph Convolutional Network (TAGCN) [29] learns a structured interaction representation that captures transmission direction, timing, and interaction topology.

For the chronologically ordered flow $x = \{p_1, p_2, \ldots, p_{n_x}\}$, we construct a graph from the first $\min(30, n_x)$ packets, with one node per packet. Consecutive packets with the same sending direction form a burst. Within each burst, edges connect consecutive packets in temporal order. Between adjacent bursts, edges connect their first nodes and their last nodes, capturing transmission continuity and direction changes. This procedure produces an undirected flow interaction graph $G(x) = (V, E)$, where V and E denote the node and edge sets. Fig. 3-2 illustrates burst segmentation, graph construction, node features, and neighborhood aggregation.

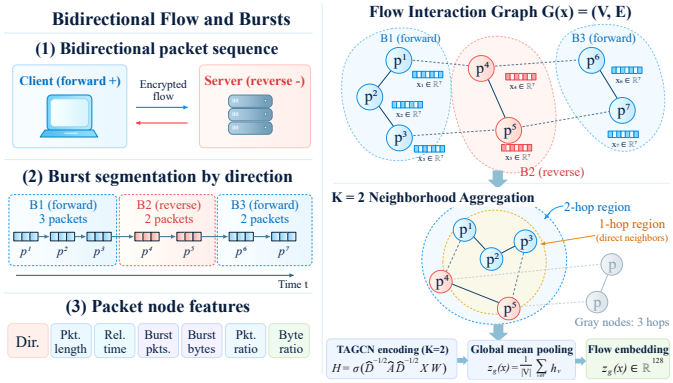


Fig. 3-2. Flow interaction graph construction and neighborhood aggregation.

Each node has seven interaction features: sending direction, packet length, elapsed time relative to the first packet, the packet count and total byte count of its burst, and the ratios of the current burst's packet and byte counts to those of the preceding burst. These features describe transmission attributes, local interaction scale, and changes between transmission stages. We collect packet nodes from flows in the known-class training set and compute the mean and standard deviation of each feature dimension. Dimension-wise standardization

yields the node-feature matrix $X \in \mathbb{R}^{|V|\times 7}$. Validation and testing reuse the training statistics.

TAGCN aggregates neighborhoods of multiple orders to learn structural dependencies between packets. Let A denote the adjacency matrix, with normalized form $\hat{A} = D^{-1/2}(A + I)D^{-1/2}$, where D is the degree matrix of $A + I$. The node-feature update is

$$H = \mathrm{ReLU}\left(\sum_{k=0}^{2} \hat{A}^k X W_k + b\right), \tag{3}$$

Here, W_k and b are learnable parameters, and $H \in \mathbb{R}^{|V|\times 128}$ combines node attributes and multihop topology. Global mean pooling then produces the flow-level graph feature:

$$z_g(x) = \frac{1}{|V|}\sum_{u\in V} H[u,:] \in \mathbb{R}^{128}, \tag{4}$$

The resulting 128-dimensional representation $z_g(x)$ serves as input to multiview adaptation and fusion.

3) Byte-Image Branch: Packet headers and payloads exhibit hierarchical byte organization beyond byte-sequence patterns and packet interactions. Direct concatenation of raw bytes can allocate excessive representation space to longer packets and discard structural information from other packets. Following YaTC's Multi-Level Flow Representation (MFR) [6], we organize raw bytes into a two-dimensional matrix with a fixed layout. Hierarchical encoding captures byte-, packet-, and flow-level structure.

For a bidirectional flow x, we select the first five packets and retain an 80-byte header region and a 240-byte payload region per packet, padding missing content with zeros. The fixed header and payload positions form an 8×40 packet-level matrix. Stacking these matrices in chronological order yields the 40×40 flow-level MFR matrix:

$$B(x) = \mathrm{Stack}(M_1, M_2, \ldots, M_5) \in \mathbb{R}^{40\times 40}, \tag{5}$$

Here, M_i is the two-dimensional byte matrix of packet i, containing separate header and payload regions. Fixed spatial positions for bytes at different levels preserve within-packet structure and flow-level organization. We normalize byte values to $[-1, 1]$ before encoding.

The pretrained YaTC encoder partitions the MFR matrix into nonoverlapping patches and constructs initial features through linear projection and positional embeddings. Packet-level attention first extracts byte relationships within packets, and row pooling (RP) aggregates them into row-level representations:

$$U_p = E_{\mathrm{PL}}(B(x)), \qquad U_r = \mathrm{RP}(U_p), \tag{6}$$

Here, E_{PL} comprises patch embedding and packet-level attention, U_p is the packet-level representation, and U_r is the representation after row pooling.

Flow-level attention then models dependencies across packets, and column pooling (CP) produces the flow-level representation:

$$U_f = E_{\mathrm{FL}}(U_r), \qquad z_y(x) = \mathrm{CP}(U_f) \in \mathbb{R}^{192}, \tag{7}$$

Here, E_{FL} is the flow-level attention encoder, U_f is its output, and $z_y(x)$ is the final 192-dimensional byte-image feature. We initialize the encoder with pretrained YaTC parameters and fine-tune it with supervision on the known-class training set. The resulting multilevel byte features feed the adaptation and fusion module.

C. Gaussian Adaptation and Deterministic Feature Fusion

The three views capture byte context, packet interactions, and multilevel byte organization, but their feature dimensions and numerical scales differ. We first standardize each encoded view feature $z_v(x)$ using known-training-set statistics, then map it into a common latent space with a view-specific Gaussian adapter. Let $v \in \{t, g, y\}$ index the byte-sequence, flow-interaction-graph, and byte-image views, respectively. The conditional distribution of each latent representation is

$$q_v(u \mid x) = \mathcal{N}\left(\mu_v(x), \mathrm{diag}\left(\exp(a_v(x))\right)\right), \tag{8}$$

Here, $\mu_v(x)$ and $a_v(x)$ denote the latent mean and log variance, respectively. All views share latent dimension d_u.

Following the variational regularization principle used in the deep variational information bottleneck [30], Gaussian adaptation jointly optimizes class discrimination and latent-distribution constraints. During training, reparameterized sampling from $q_v(u \mid x)$ produces latent representations. Classification cross-entropy preserves information relevant to known classes, while KL divergence from the standard normal distribution regularizes the latent distribution. The adaptation objective is

$$\begin{aligned}\mathcal{L}_{\mathrm{adapt}} = \frac{1}{3M}\sum_{i=1}^{M}\sum_{v\in\{t,g,y\}}\Big[&\mathcal{L}_{iv}^{\mathrm{CE}} \\ &+ \frac{\beta}{d_u} D_{\mathrm{KL}}\left(q_v(u \mid x_i)\,\|\,\mathcal{N}(0, I)\right)\Big].\end{aligned} \tag{9}$$

Here, M is the batch size. For sample i and view v, $\mathcal{L}_{iv}^{\mathrm{CE}}$ is the classification cross-entropy between the true label and the prediction of a view-specific classifier operating on the reparameterized latent sample. The coefficient β controls distribution regularization. The classification term preserves class differences, while the KL term constrains deviations from the standard normal distribution.

To eliminate randomness from latent sampling in distance calculations, we construct a deterministic fused representation from the latent means. We scale all three means by the same coefficient, concatenate them, and apply a learnable mapping to obtain fused features. A classification head outputs known-class scores:

$$\begin{aligned} r(x) &= \mathrm{concat}\left(\frac{\mu_t(x)}{3}, \frac{\mu_g(x)}{3}, \frac{\mu_y(x)}{3}\right), \\ h(x) &= F_\varphi(r(x)) \in \mathbb{R}^{128}, \\ \ell(x) &= g_\psi(h(x)) \in \mathbb{R}^{C}. \end{aligned} \tag{10}$$

Here, $r(x)$ is the concatenated representation, F_φ learns combinations of features across views, and g_ψ is the known-class classification head. The score $\ell_c(x)$ corresponds to class c. Global and local empirical support modeling also uses

$h(x)$, placing known-class classification and unknown-class discrimination in the same representation space.

D. Global and Local Empirical Support Modeling

Known-class classification distinguishes classes, whereas unknown-class discrimination must also assess a test sample's relationship to known training samples. Mean-based class prototypes are a standard reference in metric-based classification [31]. They summarize the overall location of training representations but cannot fully characterize within-class distributions. A sample near a centroid may still lie far from actual training samples. We therefore combine class centroids and within-class nearest neighbors in the fused space to characterize deviations in global position and local proximity. In the learned, fixed representation space, known training samples define a reference set for each class:

$$\begin{aligned} \mathcal{T}_c &= \{h(x_i) : y_i = c, (x_i, y_i) \in \mathcal{D}_{\text{tr}}\}, \\ N_c &= |\mathcal{T}_c|. \end{aligned}$$

Here, $\mathcal{T}_c$ is the training representation set of known class c, and N_c is its size. The mean representation b_c serves as the empirical centroid, summarizing the class's overall position. For a test flow x, distances to all class centroids determine the nearest-centroid class $c^*(x)$. The corresponding squared Euclidean distance defines the global distance $d_G(x)$:

$$\begin{aligned} b_c &= \frac{1}{N_c} \sum_{h_i \in \mathcal{T}_c} h_i, \\ c^*(x) &= \arg\min_{c \in \mathcal{C}_K} \|h(x) - b_c\|_2^2, \\ d_G(x) &= \|h(x) - b_{c^*(x)}\|_2^2. \end{aligned} \tag{11}$$

The global distance measures deviation from the nearest class centroid. Nonparametric nearest-neighbor distances provide a sample-based criterion for OOD detection [17]. Here, we restrict the neighborhood to the nearest-centroid class and use the mean neighbor distance. Specifically, we select the k representations nearest to $h(x)$ in Euclidean distance from $\mathcal{T}_{c^*(x)}$. These representations form the local neighborhood $\mathcal{N}_k(x)$. The local distance is their mean distance to the test representation:

$$d_L(x) = \frac{1}{k} \sum_{h_j \in \mathcal{N}_k(x)} \|h(x) - h_j\|_2, \qquad \mathcal{N}_k(x) \subseteq \mathcal{T}_{c^*(x)}. \tag{12}$$

A larger $d_L(x)$ indicates greater separation from the selected class's training neighborhood and thus weaker local empirical support. Global and local distances provide complementary geometric evidence of deviation from the class's overall location and actual training neighborhood. The nearest-centroid class $c^*(x)$ selects the class for neighborhood construction. The classification head still determines the final known-class label of an accepted sample; the two class assignments need not agree.

E. Robust Standardization and Open-Set Decision

The global distance $d_G(x)$ is a squared Euclidean distance to a centroid, whereas $d_L(x)$ is a mean Euclidean distance to within-class neighbors. Their numerical scales differ, so direct combination can allow one distance to dominate. We robustly standardize them separately using the known-class validation set. For $a \in \{G, L\}$, let $\mathcal{S}_a = \{d_a(x) : x \in \mathcal{D}_{\text{val}}\}$. Its median m_a and median absolute deviation MAD_a describe location and dispersion, yielding

$$\begin{aligned} m_a &= \text{median}(\mathcal{S}_a), \\ \text{MAD}_a &= \text{median}_{u \in \mathcal{S}_a} |u - m_a|, \\ \tilde{d}_a(x) &= \frac{d_a(x) - m_a}{\text{MAD}_a + \varepsilon}. \end{aligned} \tag{13}$$

Here, the positive constant $\varepsilon > 0$ prevents division by zero. Fixed equal weights combine the standardized distances into the rejection score $s(x)$. The empirical 95% quantile of known-validation scores determines the threshold τ:

$$\begin{aligned} s(x) &= \frac{1}{2}\tilde{d}_G(x) + \frac{1}{2}\tilde{d}_L(x), \\ \tau &= Q_{0.95}\left(\{s(x) : x \in \mathcal{D}_{\text{val}}\}\right). \end{aligned} \tag{14}$$

Here, $Q_{0.95}$ denotes the empirical quantile operator. Both the standardization statistics and the rejection threshold come from the known-class validation set. A higher $s(x)$ indicates greater combined deviation from known-class centroids and training neighborhoods. Scores above the threshold trigger an unknown decision. Otherwise, the classification head predicts $\hat{c}_{\text{cls}}(x) = \arg\max_{c \in \mathcal{C}_K} \ell_c(x)$. The final open-set decision rule is

$$f(x) = \begin{cases} \hat{c}_{\text{cls}}(x), & s(x) \le \tau, \\ \perp, & s(x) > \tau, \end{cases} \tag{15}$$

Here, $\perp$ is the unified unknown-class label.

IV. Experimental Results and Analysis

We evaluate the proposed method through three research questions (RQs) concerning known-class recognition, unknown-class rejection, and component contributions:

- **RQ1: Does the proposed method offer advantages in known-class recognition?**
- **RQ2: Can the proposed method effectively identify and reject unknown services and attacks?**
- **RQ3: Do the individual views and the empirical support mechanism contribute to rejection performance?**

We design corresponding experiments to address these questions. For RQ1, Section 4.2 compares known-class recognition against existing methods on tasks derived from ISCX-VPN, ISCX-Tor, and other datasets at different class granularities. For RQ2, Section 4.3 evaluates rejection of unknown services and attacks through complete-service holdout on VNAT, class holdout on USTC-TFC2016, and unknown-attack detection on CIC-IDS2017. Section 4.5 further examines the effects of the rejection threshold and changes in the proportion and composition of unknown classes. For RQ3, Section 4.4 compares rejection scores on identical frozen representations and conducts view ablations to assess the contributions of local empirical support and individual view branches.

A. Experimental Setup

1) Datasets and Task Splits: We use bidirectional flows as samples and evaluate on ISCX-VPN [32], ISCX-Tor [33], USTC-TFC2016 [34], CIC-IDS2017 [35], and VNAT [36].

ISCX-VPN and ISCX-Tor retain six and seven original service labels for the main closed-set comparison. USTC-TFC2016 supplies three class-holdout settings. CIC-IDS2017 supports both known-class classification and unknown-attack diagnosis. VNAT supplies four service-level holdout folds. Table 4-1 lists the class and ID mappings for USTC-TFC2016, ISCX-VPN, and ISCX-Tor; Table 4-2 specifies the USTC and coarse-service class splits.

2) Baselines: We compare representative encrypted-traffic recognition methods covering closed-set classification and unknown-class detection.

- **TrafficFormer** [2]: a pretrained Transformer for encrypted-traffic representation.
- **YaTC** [6]: a masked-autoencoder traffic Transformer with multilevel flow representations.
- **Open-Detect** [11]: a variational model with Gaussian class prototypes. Its native minimum posterior-to-prototype KL score serves as the VNAT open-set baseline; the closed-set table uses the recorded corrected-paper implementation.
- **RoNeTC** [27]: multiview feature extraction and uncertainty-aware traffic classification. The closed-set experiment uses a runnable reconstruction.
- **TFE-GNN** [7]: a graph-based traffic encoder. The recorded comparison adapts it to UDP and short flows using up to eight packets.
- **Trident** [14]: class-specific one-class learners. The recorded closed-set adaptation uses 86 statistical features from up to eight packets.
- **MSP** [22]: negative maximum softmax confidence, used as an unknown score.
- **Energy** [23]: an energy score computed from classifier logits.

DES-v0 and DES-v1 are the proposed method's centroid-only and centroid-plus-neighbor rejection rules, respectively; Section III defines them.

3) Evaluation Metrics and Statistical Conventions: We use accuracy, Macro-F1, and Weighted-F1 to evaluate known-class classification. For unknown-class detection, we report AUROC, AUPRC, unknown-class F1, the unknown false acceptance rate (UFAR), and the known false rejection rate (Known FRR), treating unknown samples as positive. ROC and precision-recall curves offer complementary views of detection performance under class imbalance [37].

Let D_U denote the unknown test set, D_K^{test} the known test set, $s(x)$ the unknown-detection score, and τ the rejection threshold. Then,

$$\text{UFAR} = \frac{|\{x \in D_U : s(x) \le \tau\}|}{|D_U|},$$

$$\text{Known FRR} = \frac{|\{x \in D_K^{test} : s(x) > \tau\}|}{|D_K^{test}|}.$$

UFAR is the proportion of unknown flows incorrectly accepted as known, and Known FRR is the proportion of known flows incorrectly rejected. The primary rejection threshold is the 95th percentile (P95) of detection scores on the known-class validation set, computed using the higher quantile rule.

B. Known-Class Recognition Performance

We assess known-class discrimination on the original ISCX-VPN and ISCX-Tor service labels against six external methods, then examine coarse-service and attack-class tasks.

Our method attains Macro-F1 scores of 89.92% and 86.78% on VPN and Tor, respectively, exceeding the strongest external baseline, YaTC (87.55% and 82.66%). This is because YaTC focuses on multilevel byte representations, whereas TriGLE combines sequence dependencies, packet interactions, and byte structure through Gaussian adaptation and feature fusion. This representation difference is consistent with the observed class discrimination. Section 4.4.2 uses branch ablations to examine their contributions to unknown-class rejection.

We further assess applicability across label granularities and task types using the four-class VPN coarse-service task, five-class Tor coarse-service task, 17-class USTC-TFC2016 known-class task, and three-class balanced CIC-IDS2017 classification task. Table 4-6 summarizes the results.

TABLE 4-6
KNOWN-CLASS CLASSIFICATION OF TRIGLE (%)

Dataset	Classes	Accuracy	Macro-F1	Weighted-F1
ISCX-VPN	4	91.58	91.94	91.45
ISCX-Tor	5	89.62	89.94	89.67
USTC-TFC2016	17	98.43	98.76	98.43
CIC-IDS2017	3	99.80	99.75	99.80

Our method achieves Macro-F1 scores of 98.76% and 99.75% on USTC and CIC, respectively, and 91.94% and 89.94% on the coarse-service VPN and Tor tasks. On CIC, class-level F1 scores for BENIGN, Slowhttptest, and PortScan reach 99.84%, 99.62%, and 99.78%, respectively, indicating relatively consistent recognition across classes. The 98.76% Macro-F1 across 17 known USTC classes indicates that the TriGLE representation can discriminate feature differences among a larger number of classes.

C. Unknown-Class Rejection Performance

Overall rejection performance. We compare our method with Open-Detect on VNAT under complete-service holdout. Table 4-7 reports AUROC, unknown-class F1, UFAR, and Known FRR for four unknown-service scenarios.

TABLE 4-7
VNAT COMPLETE-SERVICE HOLDOUT RESULTS (%, P95)

Metric	Method	Comm.	File	Remote	Stream
AUROC ↑	Open-Detect	81.8	60.0	58.9	96.1
	DES-v1	**99.5**	**99.5**	**98.8**	**99.1**
Unknown F1 ↑	Open-Detect	34.2	0.3	5.7	90.6
	DES-v1	**98.5**	**99.3**	**98.0**	**97.8**
UFAR ↓	Open-Detect	78.9	99.9	97.0	13.0
	DES-v1	**0.1**	**0.6**	**3.8**	**0.0**
Known FRR ↓	Open-Detect	**3.5**	10.0	**2.4**	6.0
	DES-v1	4.9	**4.6**	2.9	**5.5**

TABLE 4-1
CLASS AND ID MAPPINGS FOR USTC-TFC2016, ISCX-VPN, AND ISCX-TOR

ID	Class	ID	Class	ID	Class	ID	Class	ID	Class
U0	Gmail	U7	SMB	U14	MySQL	V0	Chat	T0	Chat
U1	FTP	U8	BitTorrent	U15	Htbot	V1	Email	T1	Email
U2	Nsis-ay	U9	WorldOfWarcraft	U16	Tinba	V2	File Transfer	T2	File Transfer
U3	Facetime	U10	Shifu	U17	Skype	V3	P2P	T3	P2P
U4	Weibo	U11	Outlook	U18	Miuref	V4	Streaming	T4	Audio-streaming
U5	Cridex	U12	Virut	U19	Neris	V5	VoIP	T5	Video-streaming
U6	Zeus	U13	Geodo	—	—	—	—	T6	VoIP

Note: U denotes USTC-TFC2016, V denotes ISCX-VPN, and T denotes ISCX-Tor.

TABLE 4-2
EXPERIMENTAL SCENARIOS AND KNOWN/UNKNOWN CLASS SPLITS

Scenario	Known Classes (KC)	Unknown Classes (UC)
A-1	USTC-TFC2016 \ {U16}	U16 (Tinba)
A-2	USTC-TFC2016 \ {U13, U15, U16}	U13 (Geodo), U15 (Htbot), U16 (Tinba)
A-3	USTC-TFC2016 \ {U13, U15, U16, U18, U19}	U13 (Geodo), U15 (Htbot), U16 (Tinba), U18 (Miuref), U19 (Neris)
VPN-4C	{V0+V1+V5, V2, V3, V4}	None (closed-set)
Tor-5C	{T0+T1+T6, T2, T3, T4, T5}	None (closed-set)

TABLE 4-3
SAMPLE COMPOSITION FOR VNAT COMPLETE-SERVICE HOLDOUT

Held-out unknown service	Known flows	Unknown flows
Communication	21,241	2,208
File Transfer	17,579	5,870
Remote Access	9,842	13,607
Streaming	21,685	1,764

Table 4-7 shows that DES-v1 attains higher AUROC and unknown-class F1 than Open-Detect in all four folds. File Transfer reaches 99.48% AUROC and 99.34% unknown-class F1; Remote Access reaches 98.76% and 97.98%. Mean UFAR is 1.09% and mean Known FRR is 4.45%. Known FRR rises slightly for Communication and Remote Access, while UFAR falls substantially across all four services. The method therefore rejects more unknown-service flows at the stated operating point, with a service-dependent known-class trade-off.

Rejection under different unknown-class configurations. We use the A-1, A-2, and A-3 class-holdout scenarios on USTC-TFC2016 to assess changes in unknown-class counts and composition. Fig. 4-1 presents the results.

In A-2, unknown-class F1 reaches 67.41% and UFAR decreases to 46.06%. In A-3, unknown-class F1 reaches 92.31% and UFAR decreases to 9.21%. In A-1, both methods yield zero UFAR and perform similarly. Our method achieves higher unknown-class F1 and lower UFAR in both A-2 and A-3. These results show that local empirical support helps reject flows that are close to a known-class centroid but far from its observed samples. Fig. 4-2 also shows the distributions of normalized scores for known and unknown flows. Overall, our method rejects unknown traffic effectively across the tested class configurations and shows a clearer advantage over Open-Detect in scenarios with multiple unknown classes.

Class-level responses to unknown attacks. Table 4-8 reports MSP and DES-v1 AUROC and DES-v1 AUPRC and UFAR for ten unknown CIC attacks. This comparison contrasts classification confidence with empirical support scores across attack scenarios.

TABLE 4-8
RECOGNITION RESULTS FOR TEN UNKNOWN CIC ATTACKS (%, ORIGINAL TEST-POOL PROPORTIONS, P95)

Unknown attack	Unknown flows	MSP AUROC	DES-v1 AUROC	DES-v1 AUPRC	DES-v1 UFAR ↓
DoS slowloris	5,709	81.47	**99.31**	99.67	0.04
DoS GoldenEye	7,441	33.17	**97.00**	97.72	0.24
DoS Hulk	155,168	9.38	**97.38**	99.88	0.11
FTP-Patator	3,985	98.75	**99.83**	99.86	0.00
SSH-Patator	2,987	86.97	**99.13**	98.93	0.00
Web Attack – Brute Force	1,364	91.61	**99.57**	99.14	0.00
Web Attack – XSS	629	97.80	**99.75**	98.84	0.00
DoS Slowhttptest	5,096	85.97	**99.34**	99.63	0.14
DDoS	76,613	5.82	**94.90**	99.45	16.61
Bot	1,228	62.91	**94.64**	87.58	50.90

DES-v1 achieves UFAR no greater than 0.24% for eight of the ten unknown attacks, indicating effective rejection overall. Known FRR is 7.13% for every attack category. For Hulk, MSP achieves only 9.38% AUROC, whereas DES-v1 reaches 97.38%, indicating that empirical support distance can compensate for classification-confidence failures on some unknown attacks. Bot and DDoS remain challenging, with UFAR values of 50.90% and 16.61%, respectively. This variation indicates that feature overlap between attack types and known classes affects rejection performance.

DDoS achieves 99.45% AUPRC, but unknown samples account for 97.12% of its original test pool and UFAR remains 16.61%. Thus, under class imbalance, high AUPRC does not imply a low false acceptance rate at a fixed threshold; precision-recall behavior depends on class prevalence [37]. Rejection performance therefore requires joint assessment using ranking and threshold-dependent metrics. Together with VNAT and USTC, these results show effective overall unknown detection and recognition for most classes, while leaving room for improvement on unknown attacks whose feature distributions overlap those of known classes. We will address these difficult unknown classes in future work.

TABLE 4-5
CLOSED-SET CLASSIFICATION ON ORIGINAL ISCX SERVICE LABELS (%)

Task	ISCX-VPN (6 classes)					ISCX-Tor (7 classes)				
Method	Accuracy	Macro-Precision	Macro-Recall	Macro-F1	Weighted-F1	Accuracy	Macro-Precision	Macro-Recall	Macro-F1	Weighted-F1
TrafficFormer	84.91	84.80	86.12	85.66	84.21	82.54	80.56	79.68	81.49	82.73
YaTC	85.64	85.48	86.30	87.55	86.39	84.25	81.23	79.74	82.66	84.21
Open-Detect	71.45	73.29	73.13	73.01	71.19	54.52	53.16	48.50	47.69	52.38
RoNeTC	69.99	71.31	70.86	70.76	69.62	59.36	54.74	51.26	51.42	57.60
TFE-GNN	62.95	64.51	64.78	64.39	62.82	59.98	58.10	55.09	53.98	57.94
Trident	61.11	62.67	63.65	63.16	60.60	63.02	59.52	61.36	60.34	62.91
Our method	**89.21**	**89.84**	**90.05**	**89.92**	**88.96**	**88.28**	**86.65**	**83.13**	**86.78**	**88.22**

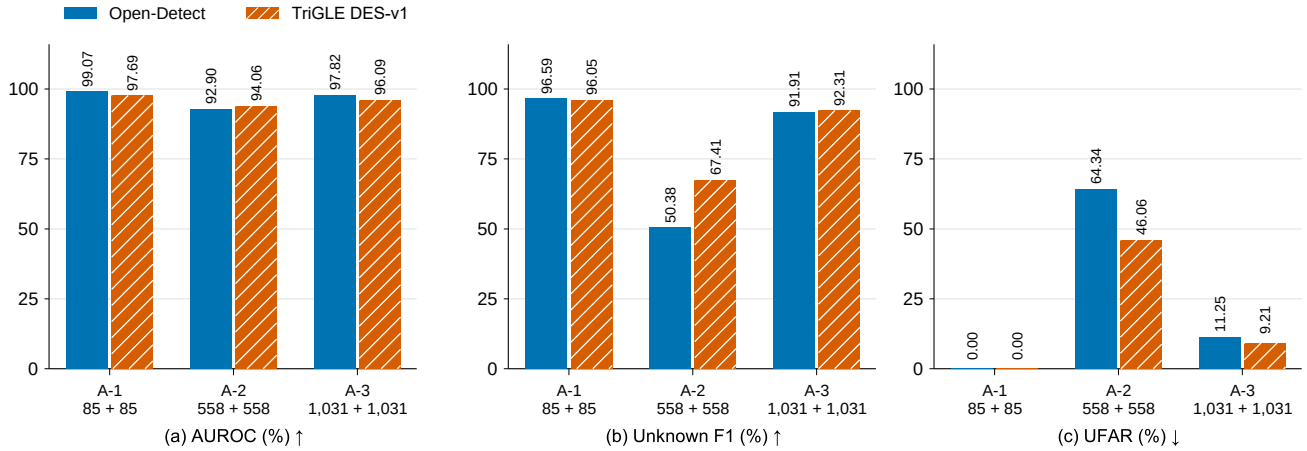


Fig. 4-1. Unknown-class detection performance under three USTC class-holdout configurations. (a) AUROC; (b) unknown-class F1; (c) UFAR.

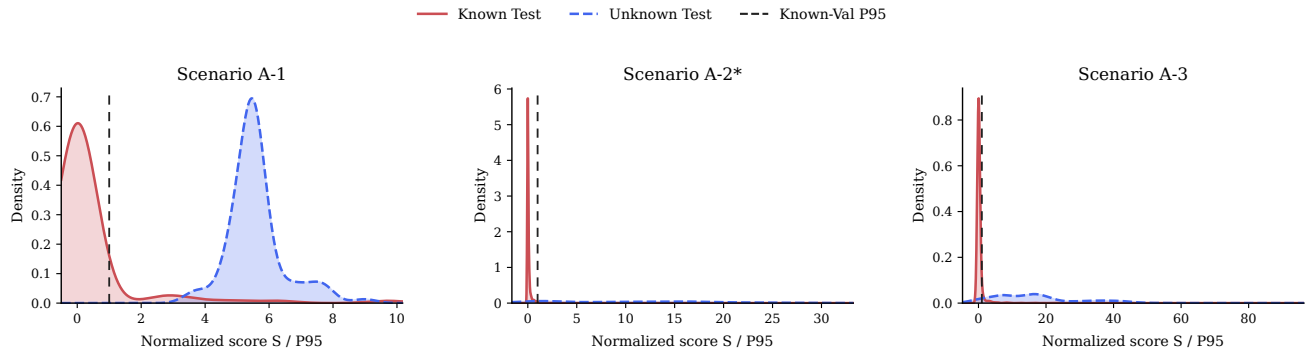


Fig. 4-2. Known and unknown score distributions in the USTC A-1, A-2, and A-3 settings.

D. Rejection Mechanism and Component Ablations

1) Roles of Global and Local Empirical Support: To separate full-system gains from the contribution of the rejection rule, we fix the encoders, fused features $h(x)$, and classification outputs $\ell(x)$ across the four VNAT folds and change only the unknown-detection score. MSP and Energy use classification outputs. DES-v0 uses the squared distance to the nearest empirical centroid. DES-v1 incorporates the mean Euclidean distance to the 10 nearest neighbors within the nearest-centroid class. Each score independently calibrates its threshold on the known-class validation set. Table 4-9 presents the results.

TABLE 4-9
REJECTION SCORES ON IDENTICAL FROZEN REPRESENTATIONS (%, EQUAL-WEIGHT MEAN OVER FOUR FOLDS, ORIGINAL TEST-POOL PROPORTIONS, P95)

Rejection score	AUROC	AUPRC	Unknown F1	UFAR	Known FRR
MSP	91.83	96.66	90.55	14.63	4.93
Energy	91.73	96.58	89.96	15.40	4.87
DES-v0	97.43	98.79	96.10	5.59	4.87
DES-v1	**99.22**	**99.50**	**98.40**	**1.09**	**4.45**

Note: DES-v0 uses empirical centroids only; DES-v1 combines global and local support. Higher AUROC, AUPRC, and Unknown F1 are better; lower UFAR and Known FRR are better.

Local nearest-neighbor information further improves unknown detection on fixed representations. Compared with DES-v0, DES-v1 increases mean AUROC from 97.43% to

99.22%, reduces UFAR from 5.59% to 1.09%, and reduces Known FRR from 4.87% to 4.45%. For Remote Access, AUROC rises from 94.47% to 98.76%; for Communication, it rises from 98.24% to 99.54%. This comparison directly supports the combination of global and local empirical support in Section III: neighborhoods of actual samples provide rejection information beyond class means. Fig. 4-4 visualizes these per-service differences.

The File Transfer fold further illustrates the trade-off between the two error rates. DES-v1 reduces Known FRR from 5.03% to 4.61%, while UFAR increases from 0.36% to 0.56%. Local support therefore primarily improves ranking across the four folds and average acceptance behavior. A specific operating point still requires joint assessment of false rejection of known services and false acceptance of unknown traffic.

2) View Contributions: We ablate the FIG, YaTC, and TrafficFormer branches individually under the same four-fold VNAT configuration to assess each branch's contribution to the final representation. The encoders, Gaussian adapters, and normalization parameters remain frozen. All configurations retain the same fusion-head architecture and training settings for comparability. Table 4-10 reports closed-set classification and unknown-detection performance.

TABLE 4-10
VNAT VIEW ABLATIONS UNDER THE SAME CONFIGURATION (%, EQUAL-WEIGHT MEAN OVER FOUR FOLDS, DES-V1, ORIGINAL TEST-POOL PROPORTIONS, P95)

Configuration	Known Macro-F1	AUROC	AUPRC	Unknown F1	UFAR	Known FRR
TriGLE (full)	99.87	99.22	99.50	98.40	1.09	4.45
Without FIG	97.08	97.45	97.37	97.24	1.41	3.49
Without YaTC	96.56	95.92	96.75	89.83	14.80	4.91
Without TrafficFormer	95.21	96.35	94.81	94.14	8.53	4.81

Note: Higher Known Macro-F1, AUROC, AUPRC, and Unknown F1 are better; lower UFAR and Known FRR are better.

Removing any branch degrades some key detection metrics in Table 4-10. Removing YaTC decreases unknown-class F1 from 98.40% to 89.83% and increases UFAR from 1.09% to 14.80%, indicating the importance of multilevel byte representations for separating unknown traffic from known classes. Removing TrafficFormer decreases AUPRC from 99.50% to 94.81% and unknown-class F1 to 94.14%, indicating that contextual information in byte sequences complements known-class discrimination and unknown detection. Removing FIG decreases AUROC and AUPRC to 97.45% and 97.37%, respectively, decreases unknown-class F1 to 97.24%, and increases UFAR to 1.41%. These results indicate that the flow interaction graph provides structural information for traffic discrimination and unknown detection by modeling relationships between packets. Overall, the three views provide complementary information that jointly improves known-class recognition and unknown-class rejection.

E. Sensitivity to the Rejection Threshold and Test Distribution

1) Effect of the Rejection Threshold: The preceding results use the fixed P95 operating point. We retain the VNAT network, training support bank, and score, and additionally use P90 and P99 of the known-class validation scores to examine false acceptance and false rejection. Table 4-11 uses the existing four-fold results, with P95 remaining the primary operating point.

TABLE 4-11
SENSITIVITY TO VALIDATION-QUANTILE THRESHOLDS ON VNAT (%, DES-V1, EQUAL-WEIGHT MEAN OVER FOUR FOLDS, ORIGINAL TEST-POOL PROPORTIONS)

Validation quantile	Unknown F1 ↑	UFAR ↓	Known FRR ↓
P90	97.26	0.91	10.02
P95	98.40	1.09	4.45
P99	85.94	20.98	1.23

The validation quantile directly controls the trade-off between acceptance of known samples and missed detection of unknown samples. P90 reduces mean UFAR to 0.91% but increases Known FRR to 10.02%. P99 reduces Known FRR to 1.23% but increases UFAR to 20.98%. A more permissive threshold increases known acceptance and may also increase missed unknown detections. All three operating points share the same score, so AUROC and AUPRC remain unchanged. Fixed-threshold error rates thus complement ranking metrics by characterizing acceptance behavior.

2) Unknown Proportion and Class Composition: Threshold sensitivity examines acceptance criteria under a fixed distribution, whereas mixed testing examines sample composition. From the frozen score pools for the ten CIC attacks, we draw 2,000 flows per trial and repeat the experiment using 20 deterministic seeds. With equal known and unknown proportions and equal representation of the ten unknown classes, DES-v1 achieves mean AUROC of 98.10%, unknown-class F1 of 93.12%, UFAR of 6.71%, and Known FRR of 7.07%. The model and P95 threshold remain frozen throughout.

Fig. 4-5 shows AUROC, AUPRC, UFAR, and Known FRR at five proportions of unknown traffic.

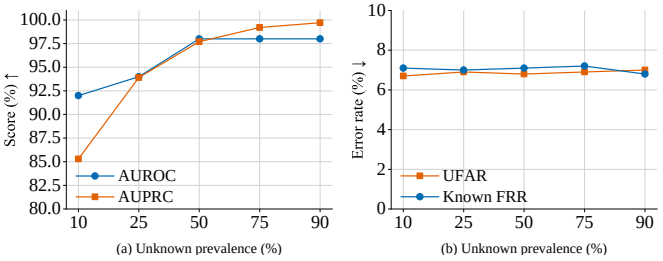


Fig. 4-5. Prevalence sensitivity across five equally spaced settings. (a) AUROC and AUPRC. (b) UFAR and Known FRR.

Ranking performance and conditional error rates remain relatively stable as the proportion of unknown samples changes. With the ten unknown classes balanced internally and the known-to-unknown ratio varying from 9:1 to 1:9, Fig. 4-5 shows AUROC ranging from approximately 92.0% to 98.0% and UFAR from approximately 6.7% to 7.0%. AUPRC increases from approximately 85.3% to 99.7% as the unknown proportion rises. At a fixed 1:1 ratio, mixing only Bot and DDoS attack samples increases UFAR to 33.84%. Changes in the composition of normal traffic and PortScan also affect

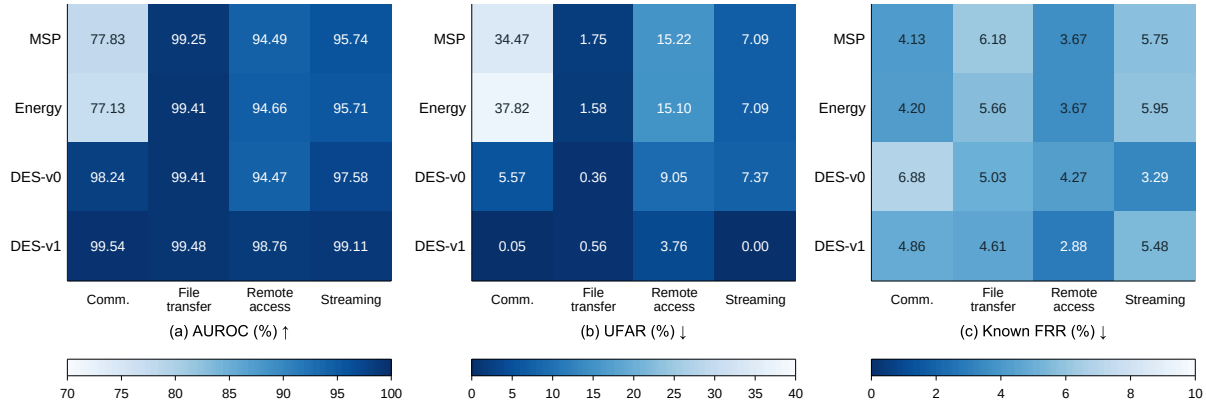


Fig. 4-4. Ablation of rejection scores on identical TriGLE representations. Columns denote the four unknown services, and rows denote the four scores. The figure reports AUROC, UFAR, and Known FRR in sequence; all cells are percentages.

Known FRR. Together with the per-attack results, these findings indicate that the detector is more sensitive to unknown-class composition than to the overall proportion of unknown samples.

V. Conclusion

The experiments show that TriGLE classifies known traffic and rejects many unknown services and attacks under the stated protocols.

Limitations: We validate the proposed method on multiple public datasets and under different unknown-class holdout settings, but dataset provenance and traffic distributions constrain the experiments. The datasets differ in capture environments, application types, and traffic composition. Unknown-class feature distributions also strongly affect rejection performance. The model still falsely accepts some unknown traffic whose features resemble those of known classes.

Future work: We will expand dataset and experimental coverage through cross-dataset and cross-network-environment generalization tests to assess adaptation to different traffic distributions. To address feature overlap between unknown and known classes, we will investigate more effective local feature modeling and rejection strategies for difficult unknown classes. We will also examine the effects of packet padding, segmentation, and traffic perturbations on multiview representations and explore detection methods that depend less on raw bytes to improve robustness in complex network environments.

References

[1] X. Lin et al., "ET-BERT: A Contextualized Datagram Representation with Pre-training Transformers for Encrypted Traffic Classification," WWW, 2022, doi: 10.1145/3485447.3512217.

[2] G. Zhou et al., "TrafficFormer: An Efficient Pre-trained Model for Traffic Data," IEEE S&P, 2025, doi: 10.1109/SP61157.2025.00102.

[3] M. Shen et al., "Accurate Decentralized Application Identification via Encrypted Traffic Analysis Using Graph Neural Networks," IEEE TIFS, 2021, doi: 10.1109/TIFS.2021.3050608.

[4] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning," ACM CCS, pp. 1928–1943, 2018, doi: 10.1145/3243734.3243768.

[5] X. Y. Chen et al., "MIETT: Multi-Instance Encrypted Traffic Transformer for Encrypted Traffic Classification," AAAI, 2025, doi: 10.1609/aaai.v39i15.33748.

[6] R. Zhao et al., "Yet Another Traffic Classifier: A Masked Autoencoder Based Traffic Transformer with Multi-Level Flow Representation," AAAI, 2023, doi: 10.1609/aaai.v37i4.25674.

[7] H. Zhang et al., "TFE-GNN: A Temporal Fusion Encoder Using Graph Neural Networks for Fine-grained Encrypted Traffic Classification," WWW, 2023, doi: 10.1145/3543507.3583227.

[8] S. Vaze, K. Han, A. Vedaldi, and A. Zisserman, "Open-Set Recognition: A Good Closed-Set Classifier Is All You Need?" ICLR, 2022.

[9] J. Zhang et al., "Autonomous Unknown-Application Filtering and Labeling for DL-based Traffic Classifier Update," INFOCOM, 2020, doi: 10.1109/INFOCOM41043.2020.9155292.

[10] Q. Yang et al., "End-to-End Open-Set Semi-Supervised Learning for Fine-Grained Encrypted Traffic Classification," IEEE TIFS, 2026, doi: 10.1109/TIFS.2026.3653575.

[11] Q. Meng et al., "Detection of Unknown Attacks Through Encrypted Traffic: A Gaussian Prototype-Aided Variational Autoencoder Framework," IEEE TIFS, 2025, doi: 10.1109/TIFS.2025.3612141.

[12] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boult, "Toward Open Set Recognition," IEEE Trans. Pattern Anal. Mach. Intell., vol. 35, no. 7, pp. 1757–1772, 2013, doi: 10.1109/TPAMI.2012.256.

[13] A. Bendale and T. E. Boult, "Towards Open Set Deep Networks," CVPR, pp. 1563–1572, 2016, doi: 10.1109/CVPR.2016.173.

[14] Z. Zhao et al., "Trident: A Universal Framework for Fine-Grained and Class-Incremental Unknown Traffic Detection," WWW, 2024, doi: 10.1145/3589334.3645407.

[15] J. Yang et al., "Conditional Variational Auto-Encoder and Extreme Value Theory Aided Two-Stage Learning Approach for Intelligent Fine-Grained Known/Unknown Intrusion Detection," IEEE TIFS, 2021, doi: 10.1109/TIFS.2021.3083422.

[16] K. Lee, K. Lee, H. Lee, and J. Shin, "A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks," Advances in Neural Information Processing Systems, vol. 31, 2018.

[17] Y. Sun, Y. Ming, X. Zhu, and Y. Li, "Out-of-Distribution Detection with Deep Nearest Neighbors," ICML, PMLR, vol. 162, pp. 20827–20840, 2022.

[18] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A Flow Sequence Network For Encrypted Traffic Classification," IEEE INFOCOM, pp. 1171–1179, 2019, doi: 10.1109/INFOCOM.2019.8737507.

[19] H. Zhang et al., "Revolutionizing Encrypted Traffic Classification with MH-Net: A Multi-View Heterogeneous Graph Model," AAAI, 2025, doi: 10.1609/aaai.v39i1.32091.

[20] X. Qiu et al., "Dual-Channel Interactive Graph Transformer for Traffic Classification with Message-Aware Flow Representation," AAAI, 2025, doi: 10.1609/aaai.v39i1.32050.

[21] W. Wang, R. Arora, K. Livescu, and J. Bilmes, "On Deep Multi-View Representation Learning," ICML, PMLR, vol. 37, pp. 1083–1092, 2015.

[22] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," ICLR, 2017.

[23] W. Liu, X. Wang, J. Owens, and Y. Li, "Energy-based Out-of-distribution Detection," NeurIPS, vol. 33, 2020.

[24] H. Wang, Z. Li, L. Feng, and W. Zhang, "ViM: Out-of-Distribution With Virtual-Logit Matching," CVPR, pp. 4921–4930, 2022.

[25] Z. Zhao et al., "FOSS: Towards Fine-Grained Unknown Class Detection Against the Open-Set Attack Spectrum With Variable Legitimate Traffic," IEEE/ACM ToN, 2024, doi: 10.1109/TNET.2024.3413789.
[26] Y. Zhong et al., "RFG-HELAD: A Robust Fine-Grained Network Traffic Anomaly Detection Model Based on Heterogeneous Ensemble Learning," IEEE TIFS, 2024, doi: 10.1109/TIFS.2024.3402439.
[27] X. Wang, Y. Wang, Y. Lai, Z. Hao, and A. X. Liu, "Reliable Open-Set Network Traffic Classification," IEEE Trans. Inf. Forensics Secur., vol. 20, pp. 2313–2328, 2025, doi: 10.1109/TIFS.2025.3544067.
[28] T. van Ede et al., "FlowPrint: Semi-Supervised Mobile-App Fingerprinting on Encrypted Network Traffic," NDSS, 2020, doi: 10.14722/ndss.2020.24412.
[29] J. Du, S. Zhang, G. Wu, J. M. F. Moura, and S. Kar, "Topology Adaptive Graph Convolutional Networks," arXiv:1710.10370, 2017.
[30] A. A. Alemi, I. Fischer, J. V. Dillon, and K. Murphy, "Deep Variational Information Bottleneck," ICLR, 2017.
[31] J. Snell, K. Swersky, and R. Zemel, "Prototypical Networks for Few-shot Learning," Advances in Neural Information Processing Systems, vol. 30, 2017.
[32] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic Using Time-Related Features," ICISSP, pp. 407–414, 2016.
[33] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor Traffic Using Time Based Features," ICISSP, 2017.
[34] W. Wang et al., "Malware traffic classification using convolutional neural network for representation learning," ICOIN, pp. 712–717, 2017, doi: 10.1109/ICOIN.2017.7899588.
[35] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," ICISSP, 2018.
[36] MIT Lincoln Laboratory, "VPN/Non-VPN Network Application Traffic Dataset (VNAT)," [Online]. Available: https://www.ll.mit.edu/r-d/datasets/vpnnonvpn-network-application-traffic-dataset-vnat. Accessed: Nov. 11, 2026.
[37] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," ICML, pp. 233–240, 2006, doi: 10.1145/1143844.1143874.